Descriptive Statistics & Intro to Pandas/NumPy

Foundations of Data Analysis

Victor Wandera Lumumba, MSc.
Statistician — Data Analyst — ML Expert

Mediacrest Training College

June 18, 2026

Session Objectives

- Understand the core concepts of descriptive statistics.
- Differentiate between measures of central tendency and dispersion.
- Master the fundamentals of NumPy for fast numerical computing.
- Learn the basics of Pandas for structured data manipulation.
- Apply statistical functions to explore and summarize real-world datasets.

Deep Dive: Descriptive Statistics

"Data without statistics is just a bunch of numbers."

What is Descriptive Statistics?

Definition

Methods for organizing, displaying, and describing data by using tables, graphs, or summary measures.

- **Goal:** Summarize a massive dataset into simple, understandable insights.
- **Contrast with Inferential Statistics:** Inferential statistics draws conclusions about a population from a sample (hypothesis testing, ML predictions). Descriptive stats just describe the data you currently have.

Why Descriptive Statistics Matter

- **First Step in EDA:** Exploratory Data Analysis always begins with summary statistics.
- **Quality Control:** Helps detect data entry errors, impossible values, and missing data.
- **Baseline for ML:** You cannot evaluate if a machine learning model is performing well without understanding the baseline statistics of your target variable.
- **Stakeholder Communication:** Non-technical stakeholders understand "average" and "spread" much better than complex algorithmic outputs.

Measures of Central Tendency

Definition

A single value that attempts to describe a set of data by identifying the "center" position within that set.

- Often called measures of **location** or **average**.
- **The "Big Three":**
  1. Mean (Arithmetic Average)
  2. Median (Middle Value)
  3. Mode (Most Frequent)

The Mean (Arithmetic Average)

- The sum of all values divided by the total number of values.
- **Formula:** $\bar{x} = \frac{\sum_{i=1}^{n} x_i}{n}$
- **Characteristics:** Uses every single data point in its calculation.
- **Weakness:** Highly sensitive to **outliers** (extreme values). One billionaire in a room of teachers drastically skews the "average" wealth.

Median and Mode

- **Median:** The exact middle value when the data is sorted in ascending order.
  - **Strength:** Robust to outliers. Preferred for skewed data (e.g., income, house prices).
- **Mode:** The value that appears most frequently.
  - **Strength:** The only measure of central tendency applicable to **categorical data** (e.g., most common blood type, most sold product).

Choosing the Right Measure

- **Symmetric Data:** Mean, Median, and Mode are roughly equal. The **Mean** is generally preferred.
- **Skewed Data:** The mean is pulled toward the tail. Use the **Median** to represent the "typical" observation.
- **Categorical Data:** Use the **Mode**.
- **Pro Tip:** Always report the Median alongside the Mean for skewed distributions to give a complete picture!

Measures of Dispersion (Spread)

Definition

Describes how spread out or scattered the data points are around the center.

- **Why it matters:** Two datasets can have the exact same mean but vastly different spreads.
- **Example:** A stock that returns exactly 5% every month vs. a stock that alternates between -10% and +20%. Both average 5%, but the risk (volatility) is entirely different.
- **Key metrics:** Range, Variance, Standard Deviation, IQR.

Variance and Standard Deviation

- **Variance (σ^2):** The average of the *squared* differences from the Mean.
- **Standard Deviation (σ):** The square root of the variance.
- **Why Std Dev?** Variance is in squared units (e.g., "dollars squared"), which is hard to interpret. Standard deviation returns to the original units (e.g., "dollars").
- **Rule of Thumb:** For normal distributions, $\approx 68\%$ of data falls within ± 1 Std Dev of the mean.

Range and Interquartile Range (IQR)

- **Range:** Maximum − Minimum. Extremely sensitive to outliers.
- **Quartiles:** Values that divide sorted data into four equal parts (Q1, Q2/Median, Q3).
- **IQR (Interquartile Range):** The range of the middle 50% of the data.

$$IQR = Q3 - Q1 \tag{1}$$

- **Strength:** Robust measure of spread. Unaffected by extreme outliers. Crucial for the "1.5 × IQR" outlier detection rule.

Python for Statistical Computing

"Let's compute these metrics programmatically."

Python for Statistical Computing

- Python's built-in 'math' and 'statistics' libraries are great for basic, small-scale calculations.
- **The Problem:** They are too slow for large datasets and lack advanced multi-dimensional features.
- **The Solution:** The Data Science Stack (**NumPy** and **Pandas**).
- Built on highly optimized C-code, allowing for blazing-fast vectorized operations on millions of rows in milliseconds.

Introduction to NumPy

- **Numerical Python.**
- The foundational package for scientific computing in Python.
- **Core Object:** The ‘ndarray‘ (N-dimensional array).
- Provides a vast collection of high-level mathematical functions to operate on these arrays efficiently.
- Acts as the universal container for data exchange between other scientific libraries.

Why NumPy? (Vectorization)

- **Vectorization:** Applying operations to entire arrays without writing explicit `for` loops.
- **Benefits:** Results in cleaner, more readable, and significantly faster code.
- **Under the Hood:** NumPy uses contiguous memory blocks and leverages SIMD (Single Instruction, Multiple Data) CPU instructions.

```
# Slow Python loop vs Fast NumPy vectorization
import numpy as np

arr = np.array([1, 2, 3, 4])
squared = arr ** 2  # Element-wise squaring
print(squared)
```

Creating NumPy Arrays

```python
import numpy as np

# From a Python list
data = np.array([10, 20, 30, 40, 50])

# Built-in generators
zeros = np.zeros((3, 3))
ones = np.ones((2, 4))
range_arr = np.arange(0, 10, 2)

# Random data
random_data = np.random.normal(
    loc=0, scale=1, size=1000
)
```

NumPy Statistical Functions

```python
import numpy as np

ages = np.array([22, 25, 28, 22, 30, 35, 40])

print(np.mean(ages))
print(np.median(ages))
print(np.std(ages))
print(np.var(ages))
print(np.percentile(ages, 75))
```

- **Pro Tip:** Use the `axis` parameter for 2D arrays.
- `axis=0` performs operations down columns.
- `axis=1` performs operations across rows.

Introduction to Pandas

"NumPy is for math; Pandas is for data."

What is Pandas?

- **Python Data Analysis Library**
- Built on top of NumPy.
- Provides high-level, flexible, and expressive data structures for working with structured (tabular) data.
- Industry standard for data wrangling, cleaning, and preparation.

Core Pandas Objects

- **Series:** A one-dimensional labeled array. Similar to a single Excel column.
- **DataFrame:** A two-dimensional labeled data structure. Similar to an Excel worksheet or SQL table.

Note

A DataFrame can be viewed as a collection of Series objects sharing the same index.

Loading Data into Pandas

```python
import pandas as pd

# Reading from CSV
df = pd.read_csv('patient_data.csv')

# Reading from Excel
df_excel = pd.read_excel(
    'financials.xlsx',
    sheet_name='Q1'
)

# Useful parameters:
# parse_dates=['admission_date']
# na_values=['?', 'N/A']
```

Initial Data Exploration

```python
# First and last rows
df.head()
df.tail()

# Dimensions
df.shape

# Data types
df.dtypes

# Summary information
df.info()
```

The Magic of .describe()

- Automatically calculates **descriptive statistics** for all numerical columns!
- Outputs: `count`, `mean`, `std`, `min`, `25%`, `50% (median)`, `75%`, `max`.

```
# For numerical columns
df.describe()

# For categorical columns
df.describe(include=['object', 'category'])
# Outputs: count, unique, top (mode), freq

```

Selecting and Filtering Data

```
# Select a single column (Returns a Series)
ages = df['age']

# Filter rows based on conditions
high_risk = df[df['blood_pressure'] > 140]

# Combine multiple conditions (Use parentheses!)
target_group = df[(df['age'] > 50) & (df['gender'] == 'M')]

```

Handling Missing Values

- Real-world data is messy. Missing values are represented as ‘NaN‘ (Not a Number).
- We can use descriptive statistics to impute (fill) them!

```
# Count missing values per column
df.isnull().sum()

# Drop rows with any missing values
df_clean = df.dropna()

# Impute missing ages with the median
df['age'] = df['age'].fillna(df['age'].median())

```

Aggregation with GroupBy

- The **Split-Apply-Combine** strategy.
- Group data by a categorical variable and calculate statistics for each group.

```
# Average salary per department
df.groupby('department')['salary'].mean()

# Multiple aggregations at once
df.groupby('gender').agg({
    'age': 'median',
    'income': ['mean', 'std']
})

```

Correlation and Covariance

- Measuring linear relationships between variables.
- **Covariance:** Direction of the relationship.
- **Correlation (Pearson):** Strength and direction (-1 to 1). Standardized, making it interpretable.

```
# Correlation between two specific columns
df['height'].corr(df['weight'])

# Correlation matrix for all numerical columns
df.corr()

```

Best Practices for Statistical Analysis

- **Always Visualize:** Summary stats can lie (Look up Anscombe's Quartet). Always plot histograms or boxplots.
- **Check for Outliers:** Extreme values will heavily skew the mean and standard deviation.
- **Correlation $\neq$ Causation:** Just because two variables move together doesn't mean one causes the other.
- **Verify Data Types:** Ensure dates are datetime objects and categories are explicitly typed as 'category' in Pandas to save memory.

Common Pitfalls

- **Ignoring Missingness:** Deleting rows with missing data can introduce severe bias if the data is not Missing Completely at Random (MCAR).
- **Using Mean for Skewed Data:** Reporting the "average income" when the data is heavily right-skewed is misleading. Use the median.
- **Confusing Std Dev and Std Error:** Std Dev measures data spread; Std Error measures the precision of the sample mean.
- **Applying Stats to Categorical Data:** Calculating the "mean" of a zip code or phone number is mathematically possible but completely meaningless.

Hands-on Practice

1. **Exercise 1:** Load the 'mediacrest$_s$*tudents.csv*'*datasetusingPandas*.

1. **Exercise 2:** Use '.describe()' to summarize the numerical columns. What is the standard deviation of their test scores?
2. **Exercise 3:** Calculate the median and IQR of the 'study$_h$*ours*'*columnusingNumPy/Pandas*.

2. **Exercise 4:** Group the students by 'study$_m$*ethod*'(*OnlinevsPhysical*)*andfindthemean*'*final*$_g$*rade*'*foreachgroup*.

Action

Open your Jupyter Notebooks and let's code!

Q&A and Thank You!

Questions?

Victor Wandera Lumumba
Statistician — Data Analyst — ML Expert

Email: lumumbavictor172@gmail.com
Web: beyonddataanalytics.online
GitHub: Lumumba1992